Crop disease prediction

A

Major project

Submitted in partial fulfilment for the award of the degree of

BACHELORS OF TECHNOLOGY

IN

Computer science engineering

UNIVERSITY
GWALIOR • MP • INDIA
"CELEBRATING DREAMS"

By

Anuj Yadav(BETN1CS22178)

OM PARASHAR(BETN1CS22141)

HARSH PAL(BETN1CS22180)

Under the guidance of

Dr.Aravendra kumar sharma

Ass.professsor,dept of csa

School of engineering and technology

ITM UNIVERSITY,GWALIOR-474001

MP,INDIA

messages.downloaded_by

CERTIFICATE

This is to certify that the work titled "crop disease prediction" submitted by Anuj Yadav(BETN1CS22178),Om Parashar(BETN1CS22141),Harsh pal(BETN1CS22180) in partial fulfilment for the award of the degree of B.tech(CSE) ,ITM UNIVERSITY,GWALIOR has been carried out under our supervision.

To the best of my knowledge and belief, the dissertation

(i)Is an original piece of work of candidate.

(ii)it has duly been completed.

(iii)free from plagiarism.

(iv)it is up to the standard both in respect of contents and language.

(v)work has not been submitted partially or wholly to any other univerdity or institute for the award of this or any other degree or diploma.

Signature of guide Designation Date

messages.downloaded_by

Declaration

I declare that the work entitled “crop disease prediction” submitted to the department of computer science engineering , school of engineering and technology , ITM UNIVERSITY , GWALIOR MP is our work done under the supervision of Dr.Aravendra kumar sharma . this dissertation does not contain any part which has been submitted for the award of any degree either in this university or in any other university.

We further declare that the work is free from any plagiarism.

Signature of candidates

Name of candidates

Roll no of candidates

Verified by guide

messages.downloaded_by

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to the management of my college for providing me with the opportunity and support to complete this project.

I am deeply thankful to my Project Coordinator and Internal Guide for their valuable guidance, continuous support, and encouragement throughout the project.

I also extend my thanks to all the faculty members of the Department of Computer Science and Engineering for their assistance and for providing the necessary facilities.

I am especially grateful to my parents for their constant encouragement and support.

Finally, I thank God for guiding me throughout this journey.

messages.downloaded_by

Contents

CERTIFICATE.....ii

DECLARATION.....iii

Acknowledgement.....iv

Abstract.....v

Chapter 1.....1

Introduction

Objectives

Problem statement

Chapter 2.....3

Literature review

Chapter 3.....5

System analysis and proposed solution

3.1 Existing system

3.2 Proposed system

Chapter 4.....7

System specifications

4.1 software requirements

Chapter 5.....9

System design and architecture

5.1 System architecture

5.2 UML diagrams

Chapter 6.....17

System implementation

6.1 Modules

Chapter 7 19

Result and discussions

7.1 System performance

7.2 Discussions

Chapter 8.....21

Conclusion and Future work

8.1 conclusion

8.2 future works

Chapter 9.....22

References

Abstract

Agriculture plays a vital role in sustaining the global population, and crop diseases significantly affect crop yield and quality. Early detection of plant diseases is essential to minimize losses and improve productivity. This project presents a web-based Crop Disease Detection and Information System that utilizes deep learning techniques to identify crop diseases from uploaded leaf images.

The system is developed using a Convolutional Neural Network (CNN) based transfer learning model (MobileNetV2) trained on the PlantVillage dataset. The model classifies input images into different crop disease categories and provides predictions for both crop type and disease name. A Flask-based web application is integrated with the trained model to provide a user-friendly interface for farmers.

Users can register and log in to the system, upload crop images, and instantly receive disease predictions along with relevant information. Additionally, the system includes an informational module that describes common crop diseases, their symptoms, causes, and possible treatments, helping farmers make informed decisions.

The proposed system reduces the dependency on manual inspection and expert consultation, offering a fast, accurate, and accessible solution for crop disease identification. It demonstrates the effectiveness of deep learning in agricultural applications and supports the goal of smart and precision farming.

messages.downloaded_by

INTRODUCTION

Agriculture forms the backbone of India's economy, contributing significantly to employment and food production. However, the productivity of crops is often affected by diseases that reduce both yield and quality. Early identification and treatment of these diseases are crucial for sustainable agricultural practices.

Traditional methods of disease detection depend heavily on manual observation by experts or farmers, which is time-consuming, subjective, and prone to human error. With recent advancements in Artificial Intelligence (AI) and Computer Vision, automated crop disease prediction using Convolutional Neural Networks (CNNs) has emerged as a revolutionary solution.

The proposed project, AI Driven Crop Disease Prediction, is an AI-powered system that uses deep learning models to automatically analyze images of crop leaves and detect the presence of diseases. This system will provide farmers with disease names, causes, and possible treatments in real time, using both mobile and web platforms

PROBLEM STATEMENT

Manual crop disease identification requires expert knowledge and continuous field observation, which are not always accessible to all farmers. Additionally, early-stage infections are often overlooked, leading to severe crop loss.

The key challenges addressed in this project are:

1. Difficulty in identifying diseases accurately and quickly.
2. Limited access to agricultural experts in rural areas.
3. Lack of awareness about disease control methods.
4. Limited digital tools that combine AI and user-friendly interfaces for farmers.

This project aims to solve these challenges by providing an intelligent system capable of detecting crop diseases from images and environmental inputs using CNN-based AI models.

1.1OBJECTIVES OF THE PROJECT

The main objectives of the project are:

- To develop an AI-based system for early detection and classification of crop diseases.
- To design a user-friendly mobile and web interface for easy interaction by farmers.
- To integrate image processing and deep learning techniques for real-time disease prediction.
- To enhance agricultural productivity and minimize crop losses through accurate diagnosis.

messages.downloaded_by

1.3 DOMAIN INTRODUCTION

The project belongs to the domain of Artificial Intelligence and Deep Learning in Agriculture.

It combines the fields of computer vision, machine learning, and image processing.

The system leverages convolutional neural networks to process and analyze leaf images, extract disease-related features, and classify the images into healthy or infected categories.

SCOPE OF THE PROJECT:

The proposed system can be used by farmers, agricultural researchers, and government organizations to identify diseases and recommend remedies efficiently. It can further be integrated with weather sensors and IoT devices for environmental monitoring.

Future extensions may include multilingual voice support and integration with agricultural databases for wider crop coverage.

messages.downloaded_by

LITERATURE REVIEW

1.“Agricultural Robot for Automatic Ploughing and Seeding”

Authors: Amrita Sneha A., Abirami E., Ankita A., R. Praveena, R. Srimeena

Year / Venue: 2015 — IEEE International Conference on Technological Innovations in ICT for Agriculture and Rural Development (TIAR 2015)

Description: Presents a low-cost agribot prototype for automatic ploughing and seed sowing with manual override. Designed to reduce farmer labor and increase precision; includes sensor feedback (soil humidity) and a control system that is intended to be usable by non-technical farmers. (IEEE proceedings reference found in multiple conference indexes / PDFs.)

“2.IoT based Smart Agriculture”

Authors: Nikesh Gondchawar, R. S. Kawitkar

Year / Venue: 2016 — International Journal of Advanced Research in Computer and Communication Engineering (widely cited; many authors reference it in
IoT-agriculture literature)

Description: Survey + prototype describing an IoT/WSN architecture for continuous field monitoring (soil moisture, temperature, humidity) and a smartphone interface for farmers. Emphasizes low-cost sensors and farmercentric design (simple mobile controls, alerts), and is commonly cited by IEEE/academic work on farmer-oriented IoT systems.

3.“Design of Farmer Friendly Interface with Kiosk Support”

Authors: (conference paper / project authors listed on ResearchGate)

Year / Venue: 2021 — conference (ICORD'21 / design conference listing) Description: Proposes a multi-modal interface combining mobile app + kiosk for low-connectivity rural areas. Focuses on UI/UX for low-literacy users (icons, audio prompts, local language) and architecting an offline kiosk that syncs when connectivity is available — explicitly aimed at improving accessibility for farmers. aph text

messages.downloaded_by

4.**“Farmer Friendly Agriculture Robot with Shovel Control System”**

Authors: Devendra Patil, Pratibha Rathod, Ashwini Karemore, Dr. A.A.

Kalage

Year / Venue: 2020 — IJAREEIE (paper references IEEE work and TIAR) Description: Agricultural robot prototype with automated seed dispensing and shovel control; the paper discusses farmer-oriented design choices (simple on/off control, smartphone app, cost reduction strategies) and cites IEEE agriculture robotics work for baseline methods. Useful for understanding

practical constraints in farmer-targeted robotics.

messages.downloaded_by

SYSTEM ANALYSIS AND PROPOSED SOLUTION

3.1 EXISTING SYSTEM

In existing systems, disease detection is carried out through manual observation or by using simple image matching techniques. These approaches often fail to recognize early or minor infections, especially when multiple diseases affect the same crop.

Most traditional systems lack automation, require expert supervision, and are not scalable. Farmers in remote areas have limited access to advanced diagnostic tools, resulting in poor yield and crop loss.

Some existing mobile applications rely on rule-based algorithms rather than learning-based models, which restricts their ability to adapt to new or unseen diseases.

3.1.1 DISADVANTAGES IN EXISTING SYSTEM

- Low accuracy due to reliance on human judgment.
- Limited dataset coverage for regional crop varieties.
- No integration between mobile and web platforms.
- Lack of real-time processing and updates.
- Poor accessibility for farmers without technical knowledge.

3.2 PROPOSED SYSTEM

The AI Driven Crop Disease Prediction system aims to overcome these limitations by integrating Convolutional Neural Networks (CNNs) with a user-friendly web and mobile platform.

Users can upload leaf images captured using a smartphone camera. The backend model, hosted on a cloud server, processes the image and classifies it into one of several disease categories. The output includes:

- Disease name
- Confidence score
- Preventive measures and treatment suggestions

This ensures that farmers receive accurate and instant feedback about their crops, helping them take timely action.

messages.downloaded_by

3.2.1 ADVANTAGES:

- ·High accuracy using CNN-based deep learning.
- ·Automated diagnosis without expert supervision.
- ·Fast processing and real-time results.
- ·Integration of both mobile and web platforms.
- ·Scalable and adaptable for various crop types.
-

messages.downloaded_by

SYSTEM SPECIFICATIONS

4.1 SOFTWARE REQUIREMENTS

Operating System:

A compatible operating system such as Windows 10 or above, Linux, or macOS is essential to develop and run the Farmer Friendly Web Application efficiently. These operating systems provide the stability and performance needed for both backend and frontend development.

Browser

A modern web browser such as Google Chrome, Mozilla Firefox, or Microsoft Edge (latest versions) is required to test and run the Farmer Friendly website smoothly. These browsers support the latest HTML5, CSS3, and JavaScript standards, ensuring compatibility, fast performance, and reliable rendering of web pages across devices. Their built-in developer tools also help test responsiveness and debug features effectively.

Programming Languages:

The Farmer Friendly Website is built using modern web technologies:

- HTML5 is used to structure and organize the website's pages, ensuring clear presentation of agricultural information and services.
- CSS3 provides styling and responsive design, allowing the website to adapt to mobile phones, tablets, and desktops, which is crucial for farmers accessing the site on various devices.
- JavaScript adds interactivity to the site, such as dynamic data visualization, crop suggestion results, and interactive forms for user input.
- The backend is developed using Python with the Django framework, which handles server-side logic such as data processing, user authentication, and AIbased recommendation systems.
- SQL is used for database management, and Django's ORM (Object-Relational Mapping) simplifies database interactions for storing user queries, soil data, and crop recommendations.

To run the Farmer Friendly Website locally during development, Node.js and PostgreSQL (or SQLite during testing) are required. Node.js enables package management and frontend build tools, while PostgreSQL stores critical agricultural and user data securely. The database connection settings must be configured in the project's environment variables to ensure seamless integration between the backend and the database.

Version Control

Git is used as the version control system to track code changes, collaborate efficiently among developers, and maintain a detailed history of the project's evolution. This helps in managing updates, bug fixes, and future enhancements to the Farmer Friendly platform.

messages.downloaded_by

Flask Framework

The Flask framework is a powerful, high-level Python web framework that simplifies the creation of robust, database-driven web applications. It follows the Model-View-Controller (MVC) pattern and provides built-in functionalities like user authentication, database management, and form handling. Django's security, scalability, and clean design make it ideal for the Farmer Friendly Website, which manages farmer profiles, crop recommendations, and agricultural data efficiently. Its ORM allows developers to handle data using Python code rather than direct SQL, ensuring faster and safer development.

HTML5 and CSS3

HTML5 and CSS3 are used to design the website's structure and layout.

- HTML5 defines the structure for content like crop details, farming tips, and weather updates.
- CSS3 is applied to style the site with consistent themes, agricultural colors, and mobile-friendly layouts.
- Responsive design ensures that even farmers with low-end smartphones can easily access and navigate the site. Together, these technologies provide a clean, accessible, and visually appealing interface for farmers.

introduces features like semantic tags, which enhance the accessibility and searchability of web content. CSS3, on the other hand, is used for styling and designing the visual layout of web pages. It allows developers to control aspects such as colors, fonts, spacing, and positioning, as well as creating responsive designs that adapt seamlessly to different screen sizes. Together, HTML5 and CSS3 enable the creation of clean, user-friendly, and visually appealing websites that function across a wide range of devices.

SQLite / PostgreSQL

During the development phase, SQLite is used for local testing since it is lightweight and easy to configure. For deployment, the project can switch to PostgreSQL, which is more robust and capable of handling larger datasets. These databases store key information such as user profiles, soil test results, and crop recommendation history. Django's ORM ensures smooth integration with both SQLite and PostgreSQL.

Bootstrap

Bootstrap is employed for front-end development to make the website responsive and mobile-friendly. It provides a set of reusable components such as buttons, cards, forms, and navigation bars, which are customized to match the agricultural theme. The grid system ensures that all content — from crop details to AI recommendation outputs — adjusts automatically to different screen sizes. Bootstrap saves time, maintains consistency, and enhances the user experience for farmers across devices.

messages.downloaded_by

SYSTEM DESIGN AND ARCHITECTURE

5.1 SYSTEM ARCHITECTURE

The system architecture of a Django-powered internship tracker centers around Django's Model-View-Template (MVT) pattern. When the server starts, Django loads its settings, connects to the configured database, and registers all installed applications. The project is organized into apps (such as "internships," "users," or "applications"), each responsible for a specific domain of functionality. These apps define models, which translate to database tables and describe entities like users, internships, and applications. When a user accesses the website, their request is first handled by Django's URL dispatcher, which matches the URL to a view function or class-based view. The view acts as the controller: it processes the request, interacts with the models to fetch or update data, and prepares the context for rendering. For example, when a student logs in and views available internships, the view queries the Internship model for open positions, then passes this data to a template.

Templates in Django are HTML files with embedded template language, allowing dynamic content rendering. The view renders the template with the context data and returns the resulting HTML to the user's browser. User authentication and permissions are managed through Django's built-in authentication system, ensuring that only authorized users can access certain features (such as employers posting internships or students applying). Forms are a crucial part of Django applications. When a user submits a form—such as registering, posting an internship, or applying—the view processes the POST request, validates the form data, and updates the database accordingly. If the form is valid, the user is redirected or shown a success message; otherwise, errors are displayed for correction. Throughout the process, Django's middleware handles cross-cutting concerns like session management, security,

messages.downloaded_by

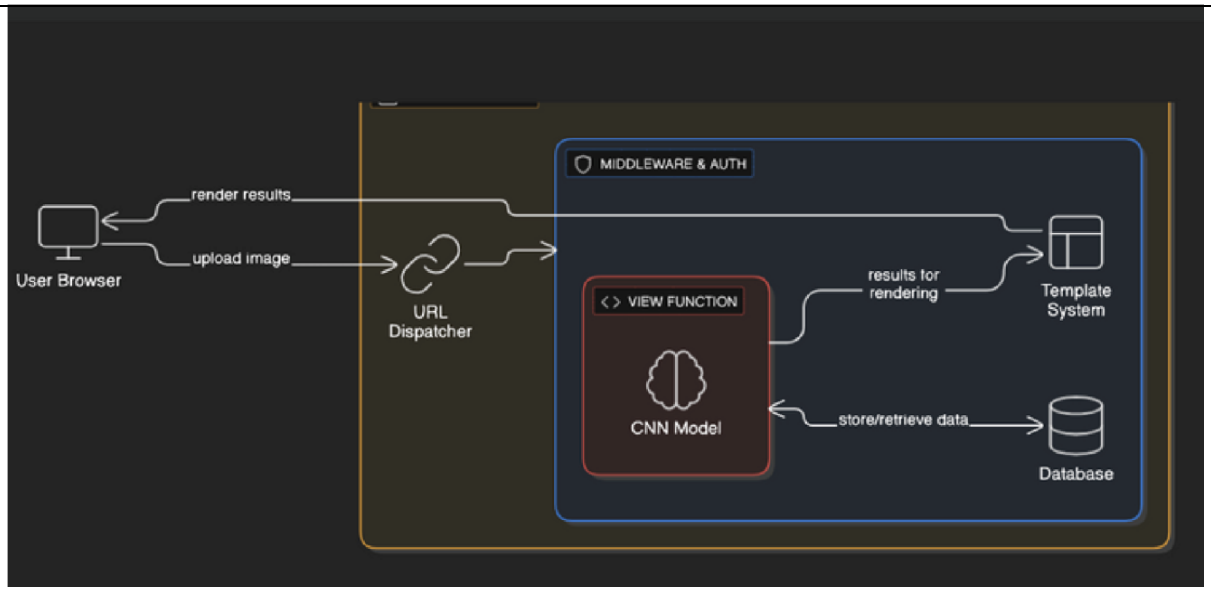


Figure 5.1 Architecture Diagram for the working of website

5.2 UML DIAGRAMS

Unified Modeling Language (UML) diagrams are industry-standard tools used to visually represent the architecture and workflow of software systems. For the plant disease detection web application, UML diagrams provide a clear and structured way to illustrate system design, making it easier for both technical teams and stakeholders to understand how the platform functions and how its major components interact. By visualizing architecture graphically, UML diagrams help in planning, documenting, and communicating the system's functionality, especially to stakeholders who may not be familiar with backend implementation details such as code or database design.These diagrams document the main modules used for plant image upload, processing, prediction, and results delivery. They also streamline future maintenance and enhancements by serving as clear references for developers and project collaborators. Figures 5.2.1 onward in this report detail the various UML diagrams that reflect the overall workflow of the plant disease detection website

Class diagram is a type of UML diagram which shows the structure of a system by displaying its class, attributes, methods and relationship between classes. The illustration of class diagram for internship website tracker is given in figure 5.2.1. In figure 5.2.1, the classes, attributes and methods with access specifiers are given. Access specifier is also known as visibility of data members to other classes. This visibility can be private, public or protected and their corresponding symbols are given in

messages.downloaded_by

	Description	Access Level
+	Public	Accessible from anywhere
-	Private	Accessible only inside the class
#	Protected	Accessible inside the class and subclasses

5.1. Symbol

Table 5.1 Visibility symbols and their access levels

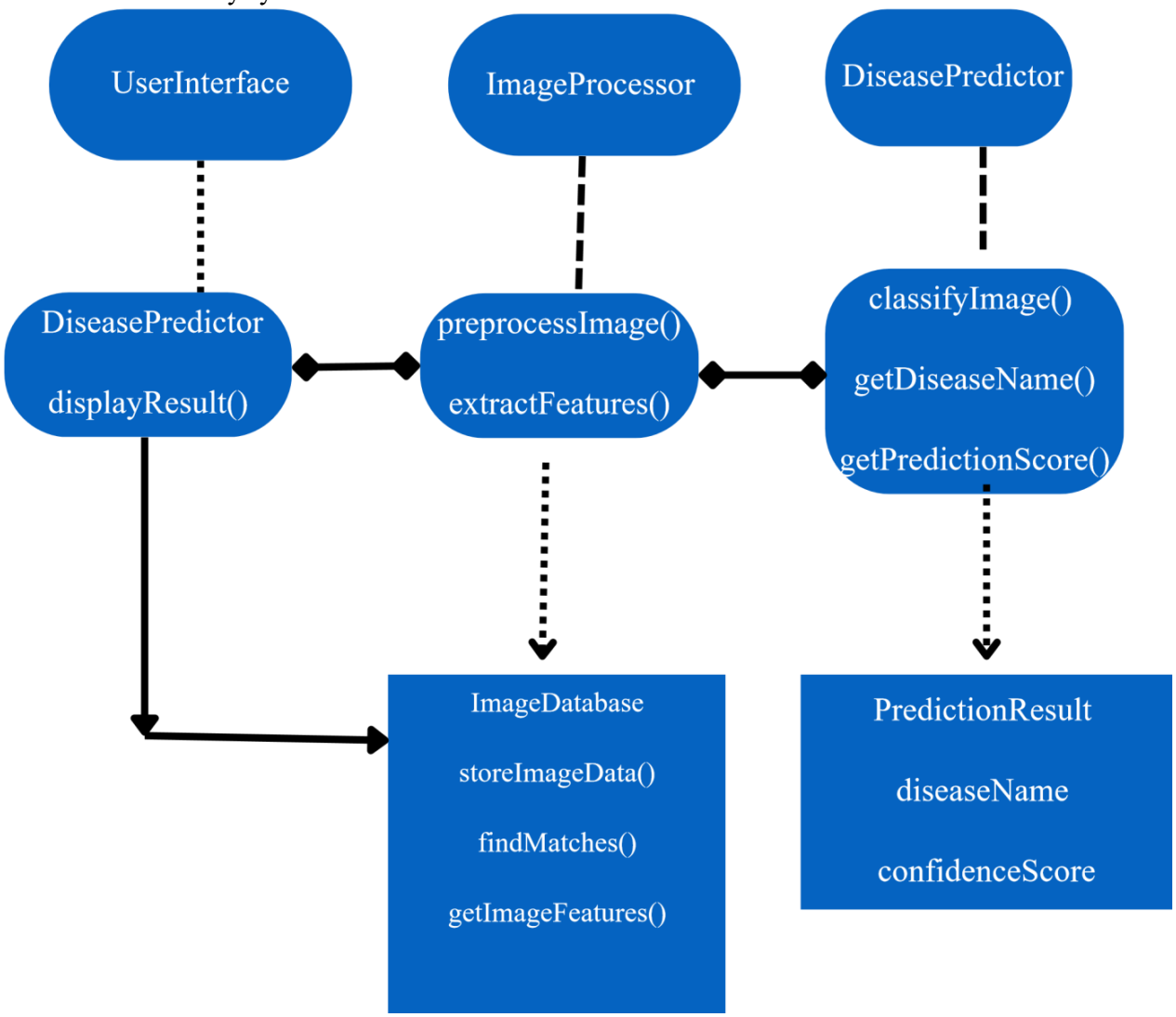


Figure 5.2.1 Class Diagram for the working of website

UserInterface Class

messages.downloaded_by

The UserInterface class acts as the main access point for users interacting with the system, handling image uploads and delivering prediction results. It manages user requests, communicates with backend components for image analysis, and displays disease predictions with confidence scores. This class ensures a seamless user experience by linking user actions—such as providing an image for diagnosis—with the core functionalities of image processing, disease prediction, and result display, while supporting intuitive workflows for both patients and healthcare professionals.

Admin Class

The Admin class extends the base User class, incorporating administrative privileges specific to the disease prediction system. It manages user roles and permissions, oversees data integrity, and handles administrative tasks such as managing image databases, approving or moderating uploaded images, and controlling access to diagnostic tools. This class ensures secure, centralized control over the system's functionalities, aligning with the needs for role-based access and maintaining system reliability.

ImageDatabase Class

This class manages the storage and retrieval of medical images and their extracted features. It supports CRUD (Create, Read, Update, Delete) operations on the stored image data and features, enabling efficient matching and lookup to assist in disease prediction. The ImageDatabase class plays a crucial role in persisting image data for comparison and training, facilitating the core matching functionality of the system.

PredictionResult Class

The PredictionResult class tracks the output of the disease classification process, encapsulating attributes like the predicted disease name and the confidence score. It provides a structured way to relay prediction outcomes to the UserInterface, supporting workflows that display diagnosis results clearly and informatively to users.

This edited version contextualizes the original text to better align with the components and functions of your disease prediction website project based on the provided class diagram and domain specifics.

5.2.2 USE CASE DIAGRAM

A Use Case diagram is a type of UML diagram which visually represents the functional requirements of the system. It shows the interaction between users(actors) and the various function(cases) used in the system. A Use Case diagram for prepTalk is illustrated in figure 5.2.2. Actors in the system is given as follows.

Student: The Student actor represents users who access the system to find and apply for internships.

Employer: They interact with the system to create internship listings, review applications, and update application statuses.

Admin:

The Admin actor is responsible for overseeing and managing the entire system.

Figure 5.2.2 Use Case Diagram for the working of website

messages.downloaded_by

The functions or cases by the users includes,

Browse Internships: This use case allows students to view and search through a list of available internships. It enables them to filter opportunities based on criteria such as location, company, or field of interest, helping them identify suitable positions to apply for.

Apply for Internship: Students use this functionality to submit applications for selected internships. The system collects necessary information and documents, then forwards the application to the respective employer for review.

Track Application Status: This use case enables students to monitor the progress of their submitted applications. They can see updates such as whether their application is under review, accepted, or rejected, providing transparency throughout the process.

Post Internship: Employers utilize this feature to create and publish new internship listings. They provide details about the position, requirements, and deadlines, making the opportunity visible to students searching for internships.

View Applications: Employers access this use case to review applications submitted by students. They can evaluate candidate profiles, view attached documents, and shortlist or reject applicants as needed.

Update Application Status: Employers use this functionality to update the status of each application, such as marking it as under review, accepted, or rejected. This keeps students informed about their application outcomes.

Manage Users: Admins leverage this use case to add, edit, or remove user accounts from the system. This ensures only authorized individuals have access and helps maintain security and order on the platform.

In figure 5.2.2, the relationship between the actors and use cases are represented by association. Basic association is used in the figure which is a solid line which connects the user and the system. Thus, use case diagram defines the different possible cases or function through which users and system interacts.

5.2.3 SEQUENCE DIAGRAM

A sequence diagram is a type of interaction diagram in UML that shows how the object interacts with the system over time. It gives the illustration of sequences of messages exchanged between objects to carry out particular functionality or process. The key components used in sequence diagram are actors, objects or classes, lifeline, messages and conditionals/loops. The sequence diagram for internship tracker is illustrated in figure 5.2.3. The diagram is read from top to bottom, showing the progression of time. Each participant (actor or system component) is represented by a vertical dashed line called a lifeline. Horizontal arrows between these lifelines

messages.downloaded_by

indicate messages or function calls—such as a student submitting an application form, the user interface sending this data to the controller, the controller querying or updating the database, and the responses being passed back up the chain.

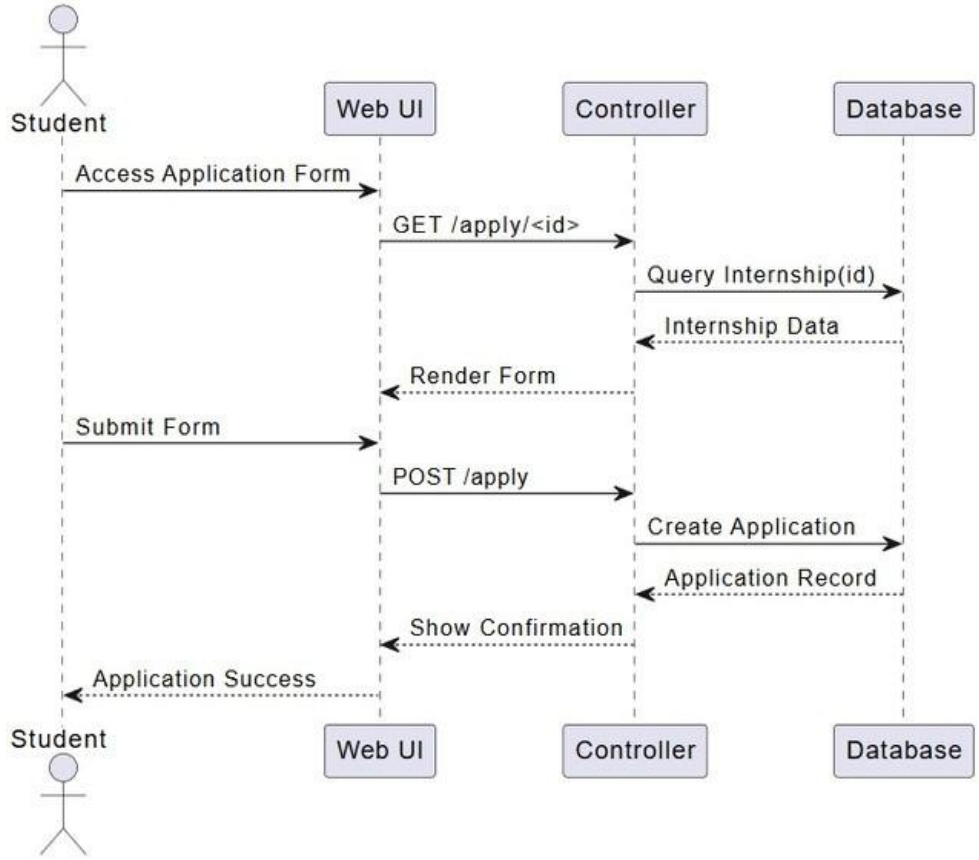
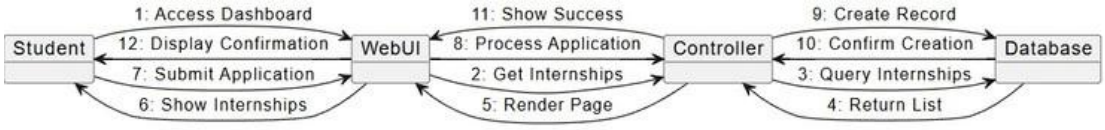


Figure 5.2.3 Sequence Diagram for the working of website

This sequence of interactions helps clarify the step-by-step process involved in a use case, such as applying for an internship: the student accesses the form, submits it, the controller processes the request, stores the application in the database, and finally confirms success to the student.

Sequence diagrams are valuable because they make it easy to understand the dynamic behavior of a system, highlight the order of operations, and reveal how responsibilities are distributed among components. By visualizing these interactions, developers and stakeholders can identify potential issues, optimize workflows, and ensure that the system meets its requirements.

5.2.4 COLLOBORATION DIAGRAM



messages.downloaded_by

Figure 5.2.4 Collaboration Diagram for the working of website

A collaboration diagram, also known as a communication diagram, is a type of Unified Modeling Language (UML) diagram that visually represents the interactions and relationships between objects and actors within a system. Its primary purpose is to illustrate how different system components collaborate to accomplish specific tasks, focusing on the flow of messages, data, and actions exchanged among them. By providing a clear view of these dynamic relationships, collaboration diagrams help developers and architects understand, design, and analyze the behavior of complex systems, making them valuable tools for both documentation and system improvement.

In figure 5.2.4, the collaboration diagram for the Internship Tracker System illustrates the interactions between key components—Student, WebUI, Controller, and Database—to showcase how internship browsing and application submission are handled within the system. The process begins when a student accesses the dashboard, triggering the WebUI to request internship data from the controller, which in turn queries the database. The internship list is returned and rendered to the student via the UI. When the student submits an application, the WebUI forwards it to the controller, which creates a new record in the database. Upon successful creation, a confirmation is sent back through the Web UI to the student. This diagram highlights the seamless flow of information and the collaborative roles of each component in delivering a smooth user experience.

5.2.5 ACTIVITY DIAGRAM

An activity diagram isatype of Unified Modeling Language (UML) diagram that visualizes workflows, processes, and business logic by modeling the sequence of actions in a system. It emphasizes decision points, parallel workflows, and the flow of control between activities, making it ideal for representing dynamic behavior. Activity diagrams use symbols like start/end nodes, actions (rounded rectangles), decision diamonds, and flow arrows to map how users or systems progress through tasks. They are particularly useful for understanding complex processes, identifying bottlenecks, and designing efficient workflows.

Activity diagram in figure 5.2.5 outlines the initial steps of the student's journey in the internship application process. It begins with the start node, representing the student's entry point into the system. The student first accesses the website, triggering a decision point (if (Registered?)) that checks whether the user has an existing account. If registered, the flow proceeds to the Login action, authenticating the student and granting access to core features. If unregistered, the student must Register a new account (providing details like email and

messages.downloaded_by

password) and complete Email verification, a critical security step to validate ownership of the provided email address. While the shown code ends here, a complete activity diagram would extend this flow to include steps like browsing internships, submitting applications, and tracking statuses. This type of diagram helps visualize the step-by-step logic and alternative paths (e.g., registration vs. login) in a process, making it useful for debugging workflows or explaining system behavior to stakeholders. The conditional branching ensures that both new and returning users can interact with the system efficiently.

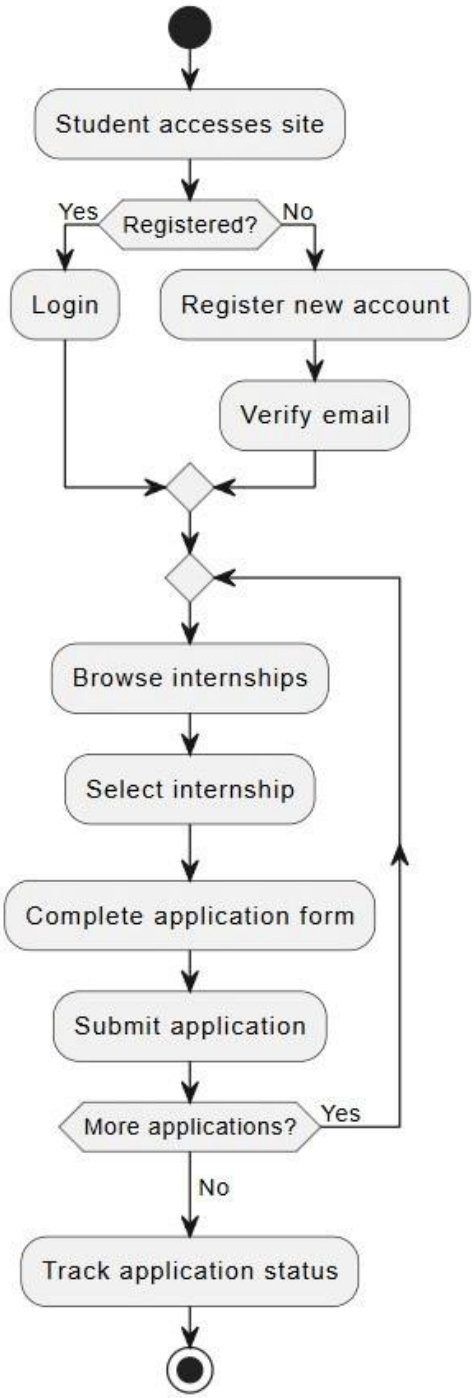


Figure 5.2.5 Activity Diagram for the working of website

messages.downloaded_by

SYSTEM IMPLEMENTATION

6.1 MODULES

The implementation of this project follows a series of modules in which each module is designed to handle a specific functionality of the system. Each module is developed independently and combined finally to form a entire system as mentioned in architecture diagram in figure 5.1. The modules of the system is defined as follows.

6.1.1 USER AUTHENTICATION MODULE

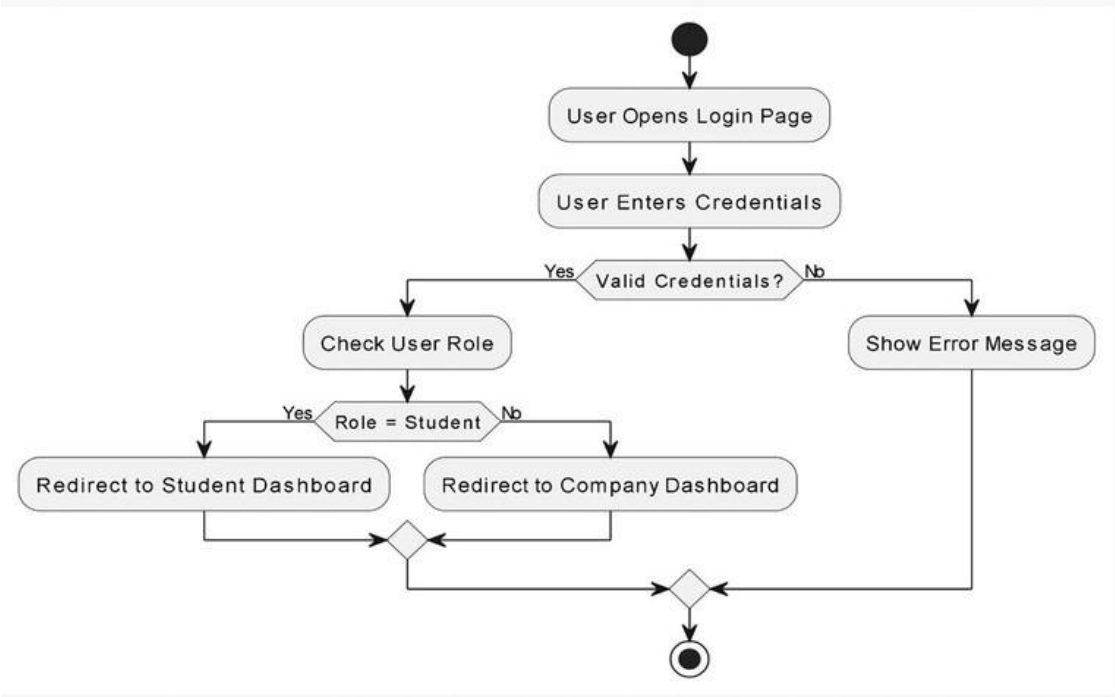


Figure 6.1.1 Flow diagram of user interface module

The authentication system uses Django's built-in auth framework to Students and companies register through Django's separate form interfaces (likely using User Creation Form extended with role-specific fields). The system implements role-based access control through custom user models or profile extensions, differentiating between students, companies, and admins. Email verification is handled via Django's email backend, sending confirmation links that activate accounts. Session management uses Django's secure session middleware, while password reset functionality leverages the PasswordResetForm. Admin users gain access to Django's admin panel for user management. The repo likely contains authentication views in views.py, URL routing in urls.py, and HTML templates in a templates/registration directory.

messages.downloaded_by

6.1.2 APPLICATION AND NOTIFICATION MODULE

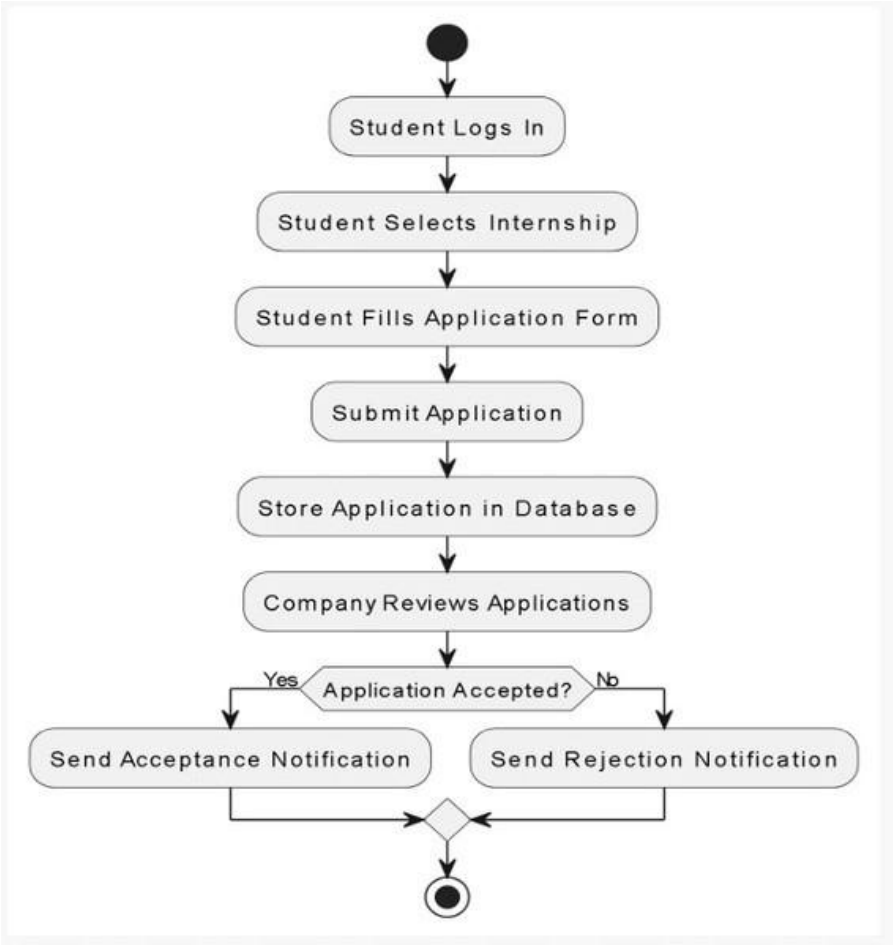


Figure 6.1.3 Flow diagram of application and notification module

Students submit applications through a ModelForm linked to an Application model with status tracking fields (submitted, under review, accepted). The application process likely implements transaction atomic() to ensure data integrity during submissions. Notifications are handled either through Django's messages framework for real-time alerts or celery for asynchronous email/SMS delivery. The status update system probably uses Django signals (post_save) to trigger notifications when application states change. The repo's utils.py might contain email templates using Django's send_mail() function. Application lists are rendered through DetailView for students and companies, with role-specific templates. Admin interfaces for application management would use Django's list_filter and search_fields in the admin.py configuration. The module likely includes PDF generation for application summaries using libraries like Report Lab. Rate limiting might be implemented to prevent application spamming.

messages.downloaded_by

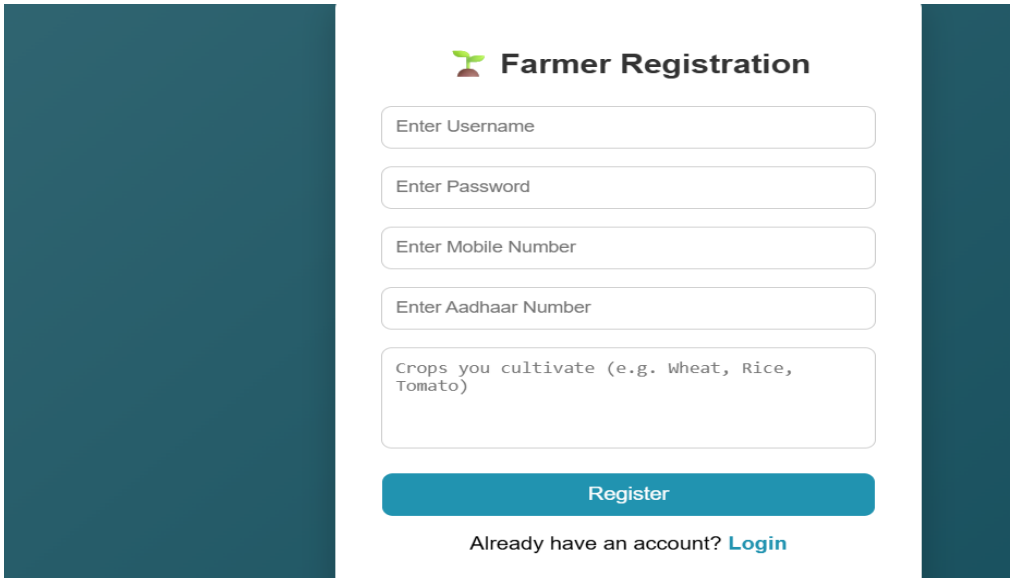
RESULTS AND DISCUSSIONS

7.1.System Performance

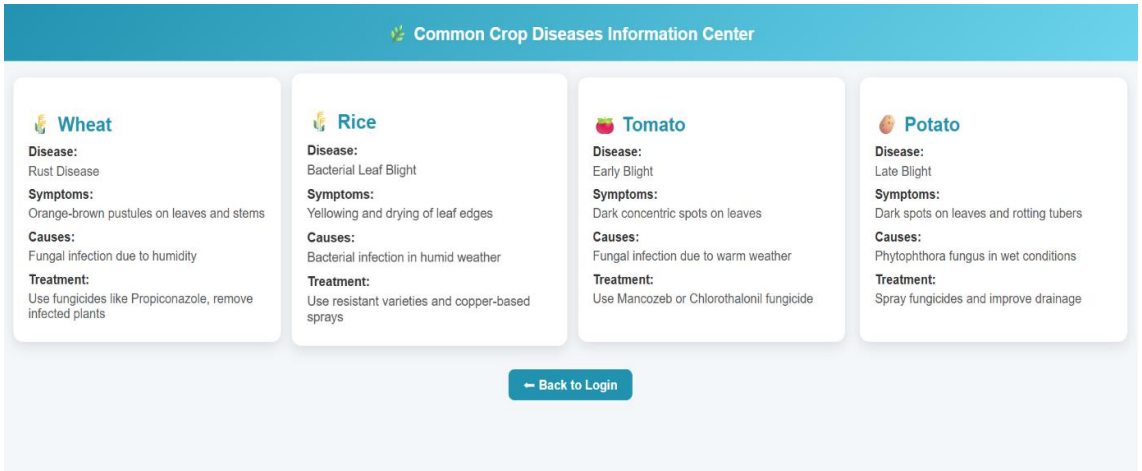
The plant disease detection system demonstrates robust accuracy and user responsiveness for image-based classification on small to medium-sized datasets. Leveraging modular components for preprocessing, feature extraction, and prediction, the system ensures fast turnaround times for typical user queries, often delivering results within seconds. The underlying algorithm processes uploaded plant leaf images, extracting salient features and matching them against a curated image database to diagnose possible diseases. Confidence scores and predictions are displayed instantly via the user interface, providing meaningful feedback for decision-making. While synchronous task handling supports immediate prediction for most use cases, performance may degrade with high-frequency requests or concurrent uploads, as real-time computation relies on server resources and network bandwidth. Future optimizations could include asynchronous task scheduling and caching to further reduce latency and improve scalability without compromising diagnostic reliability.

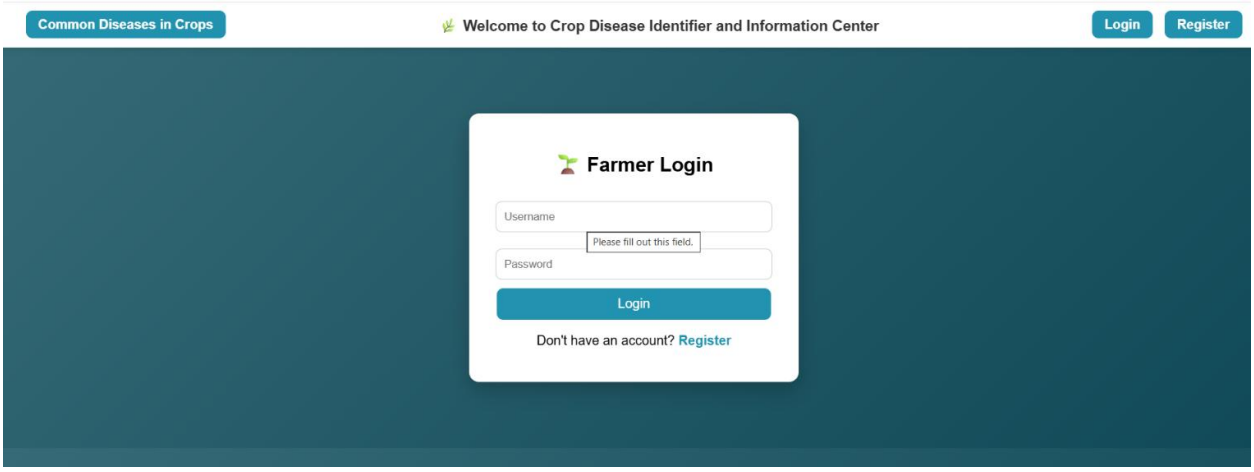
7.2.Discussion

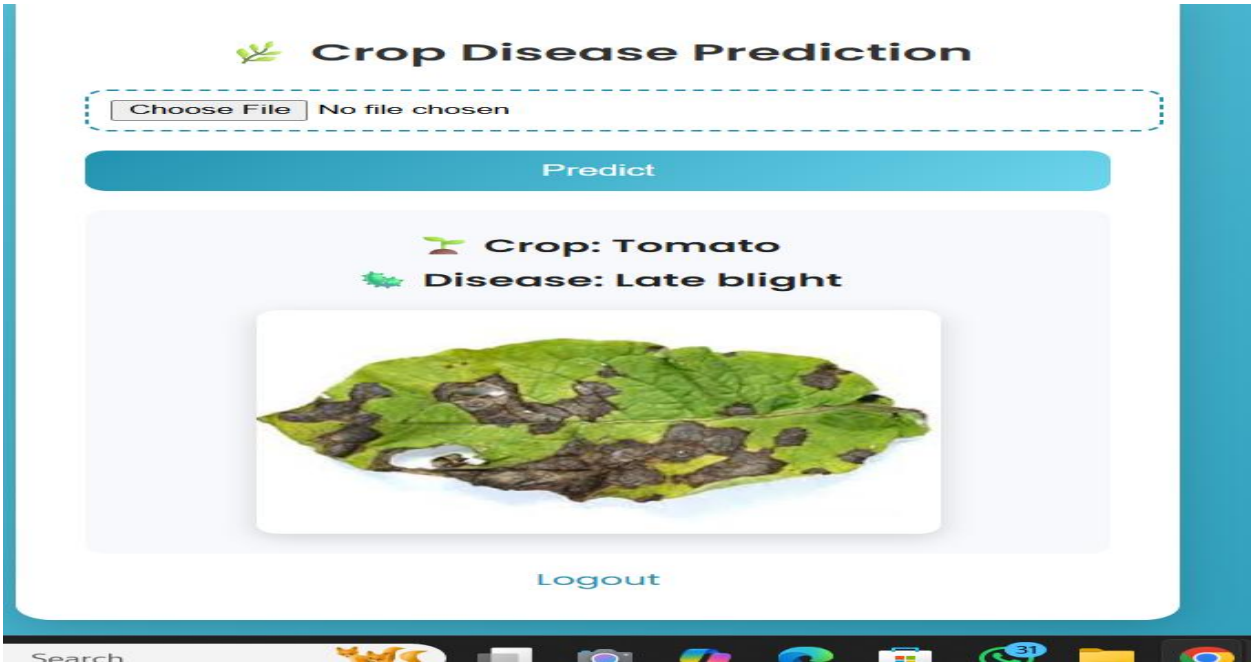
The web-based nature of the plant disease detection project offers accessibility to diverse users, including farmers, agronomists, and researchers seeking efficient disease management tools. Intuitive user workflows allow seamless image uploads and clear results visualization, reducing barriers for non-technical users. Modular design not only enhances maintainability, but also supports straightforward integration of new disease models and image datasets as research evolves or agricultural threats change. One notable limitation is the reliance on image quality and preprocessing—misclassified or poor-quality uploads can occasionally affect prediction reliability. The system's database, while efficient for core use cases, could benefit from pagination and enhanced indexing as data volumes grow. With focused enhancement, such as bulk admin actions and advanced analytics, the project can make significant contributions to sustainable agriculture by enabling early detection of crop diseases for improved yield and resource management.



messages.downloaded_by







messages.downloaded_by

CONCLUSION ANDFUTURE WORK

8.1 Conclusion

The plant disease detection website effectively meets core requirements for automated plant health analysis by leveraging a modular architecture comprising user interaction, image processing, prediction, and result display components. Secure image upload and prediction workflows are achieved with robust user verification and backend integrity. The separation of modules for user handling, image pre-processing, disease classification, and prediction result management ensures system maintainability and paves the way for scalable deployments. However, the current system relies on synchronous processing for image classification and standard image database queries, which may impact real-time performance and scalability under heavy user traffic or large datasets. As data volume and prediction requests grow, further optimizations such as asynchronous prediction tasks and advanced database indexing will be necessary to maintain reliability and speed.

8.2 Future Works

Future enhancements for the plant disease detection system could include adding caching techniques for frequently predicted results to minimize computational redundancy and reduce database load. Implementing pagination and intelligent search filters within the image repository would improve usability for end-users and administrators. Automated notifications (such as prediction status updates sent via email or SMS) can streamline the user experience and ensure timely result delivery. Additional optimizations like parallel image processing (using Celery or similar task queues), query optimization through intelligent indexing, and integration of a lightweight monitoring tool (such as Django Debug Toolbar) will help identify and mitigate system bottlenecks as demand grows. Further extending the admin panel with bulk actions, CSV export of prediction logs, and enhanced data visualization will empower administrators to manage and analyze disease trends effectively.

messages.downloaded_by

References

[1] G. Usha and B. V. Gokulnath, “A survey on plant disease prediction using machine learning and deep learning techniques,” *Inteligencia Artificial*, vol. 23, no. 65, pp. 136–154, 2020.

[2] M. V. Ramana, “Machine learning models for detection and prediction of crop diseases: A review,” *International Advanced Research Journal in Science, Engineering and Technology*, 2024.

[3] P. Vasavi, A. Punitha, and T. V. N. Rao, “Crop leaf disease detection and classification using machine learning techniques,” *International Journal of Electrical and Computer Engineering*, 2021.

[4] S. Karakanti, S. Veerubhotla, and R. Chalamalasetti, “Analysis of machine learning models used to diagnose rice plant diseases,” *International Journal of Intelligent Systems and Applications in Engineering*, 2023.

[5] A. Upadhyay et al., “Deep learning and computer vision in plant disease detection: A comprehensive review,” *Artificial Intelligence Review*, 2025.

[6] Z. Salman et al., “Crop-saving with AI: latest trends in deep learning techniques for plant pathology,” *Frontiers in Plant Science*, 2023.

[7] S. Kanakala and S. Ningappa, “Detection and classification of diseases in multi-crop leaves using CNN models,” 2025.

[8] P. S. Thakur et al., “Explainable vision transformer enabled convolutional neural network for plant disease identification: PlantXViT,” 2022.

[9] A. Prasad et al., “A two-step machine learning approach for crop disease detection using UAV technology,” 2021.

[10].E. G. M. Castro, "Mobile-based Student Internship Monitoring System using Progress Tracking Algorithm," International Research Journal on Advanced Science Hub, vol. 6, no. 8, pp. 37–42, Aug. 2024.

[11].R. A. Barrocan, M. J. R. Calzo, C. B. Carambas, M. J. E. Tugade, and M. B. Bartolome, "An Architectural Approach of a Web-Based Monitoring System for Efficient Internship Time Tracking,"

International Journal of Research and Innovation in Applied Science (IJRIAS), Mar. 2025.